

Scalability Engineering (Summer Semester 26)

Prof. Dr.-Ing. D. Bermbach, T. Rese , N. Japke, D. Baumann

Prototyping Assignment (25 Points + 5 Bonus)

In the prototyping assignment, you will design and implement a highly scalable application that builds upon the presentations and lectures that will be given in the next weeks. For this, you will be given access to funding for the Google Cloud Platform (GCP), where you can deploy your application. You're free to use any cloud service you see fit, as long as the requirements and the points from the additional information selection below are upheld. You will work in groups of two people.

The general application domain and design is left up to your group, as long as the requirements below are met.

You will have to prove in a demo that your application works as intended and that it fulfills all requirements. For this, you must submit and hold a short presentation showing your application and how it handles scalability. We will have a presentation session on **July 13th**, where each application will be presented by you.

Please also provide us with a link to the public git repository (with license and readme), where you manage your source code. Your repository needs to be documented as well: **This means that we should be able to reproduce your deployment exactly using the documentation you provided to us.**

Due until **July 12, 2026 (Sun), 23:59h**; submission via ISIS. The submission should contain the following in a .zip file:

- Presentation Slides
- Link to GitHub/GitLab (which should be open-source or accessible to us; if for some reason you do not want to open source it, our GitHub/TU-Gitlab names are (GitHub name first / GitLab):
 - timchristianrese / gov-23
 - njapke / njapke
 - dibaberlin / db2016

Requirements

1. Your application must manage some kind of state. It must consist of both stateless and stateful components and follow best practices regarding the separation of such.
2. Your application needs to be able to scale vertically and horizontally. For this, your application should be deployed in varying configurations:
 - a. A single-node instance
 - b. A 3 node-cluster (using the instance type from a.)

- c. A 5 node-cluster (using the instance type from a.)
- d. (Optional, 5 Bonus Points) Evaluate the impact that using more performant machines has on your application for the previous configurations a, b, and c, and also display these results.

You are of course open to experiment with more configurations outside of this.

You are required to track a performance metric of your application that you expect to scale with the increasing resources (such as throughput) and present these findings on a slide in your talk. You may use a load generator for this, such as K6, locust, artillery, etc.

- 3. When your application is scaled up/out, it should not be possible to overload another component (i.e., there needs to be some mitigation strategy for very high load). Your code documentation and your talk should make clear how you handle this aspect.
- 4. Implement two more strategies, which were covered either in the lecture content or during the presentations, in addition to the strategies you already implemented for previous requirements.

Deliverables

- Presentation (5 minutes) of your application, scalability results, and limitations
 - General guidelines:
 - Show your application architecture, why you chose it and how you implemented the requirements above.
 - Have 1 slide showing your scalability results regarding your chosen performance metric
 - Briefly discuss possible scalability limitations that your system might run into
 - The rest of the presentation is up to you.
- GitHub/GitLab repository link containing your application code, presentation slides and documentation. Your code and documentation should make the following points clear:
 - What is the application architecture (please provide a figure for this), and how do the components address the requirements of the task? How are stateless and stateful components split?
 - Describe the scaling-out/in and scaling-up/down process of your system. Highlight scalability and elasticity impacts that you measured
 - How are the requirements concretely implemented in your code and application? You may use screenshots of the application and/or code snippets to illustrate this.

- We are not expecting you to implement the "perfect" linearly scaling application; Address possible limitations and/or scalability issues that may occur in edge cases (if you believe that there are any)

Additional Information and Guidelines

- You can build your application using any standard programming language and tooling, but **you are not allowed to use libraries and/or built-in cloud service features to directly solve the challenges presented by requirements 1 to 4.**
 - You are allowed to use libraries for tasks such as authentication, logging, potential database access, and other things on this level.
 - **You are not allowed to use a library that automatically implements rate limiting or other scalability strategies directly.**
 - Another example are cloud services which automatically scale to your system load; these are not allowed.
- Don't plagiarize, i.e., don't copy from other students!
- Each group may apply for **ONE** GCP grant code of 50\$.
- You are allowed to use LLMs/AI for application code generation only, do not generate your presentation slides and talk using it
- Load generators do not have to be implemented yourself. You can use existing tools such as K6, Locust, artillery, etc.
- Your infrastructure must either be deployed using Terraform or deployment scripts.
- Sample applications from last semester include: A distributed URL Shortener, distributed KV store, distributed relational database, distributed AI workers, distributed Kubernetes image processing setup.